

Project Close-out Report (PCR)

Project identity

Field	Value
Full proposal / project name	OgmiosDotnet: Cardano Blockchain Events & Visualisation
Software / repository name	OgmiosDotnet.BlockchainEvents
Fund & challenge	Catalyst Fund 14 — Cardano Open: Developers (Developer Tools)
Project number	1400091
Project URL (Catalyst)	OgmiosDotnet: Cardano Blockchain Events & Visualisation
Project manager	Dave (ItsDaveB)
Date project started	January 2026
Date project completed	July 2026
Repository	github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents
Community release	v1.0.0 (badd71e)
Open source	Yes — MIT Licence
PCR PDF	close-out-report.pdf (this document as PDF for Catalyst)

Project description

OgmiosDotnet: Cardano Blockchain Events & Visualisation (repository: **OgmiosDotnet.BlockchainEvents**) is an open-source, production-oriented platform that turns live Cardano chain data into **filtered, reliable application events**.

Problem

Many Cardano applications only need a subset of on-chain activity—specific addresses, assets, metadata labels, or CIP-1694 governance/treasury transactions. Without shared infrastructure, each team must:

- Run or connect to chain sync (typically via [Ogmios](#))
- Parse every block and transaction
- Maintain language-specific Ogmios clients or WebSocket integrations
- Re-implement filtering, delivery, retry, and observability

That duplicates cost and raises the barrier for developers who simply want “the transactions that matter” over ordinary HTTP or gRPC APIs.

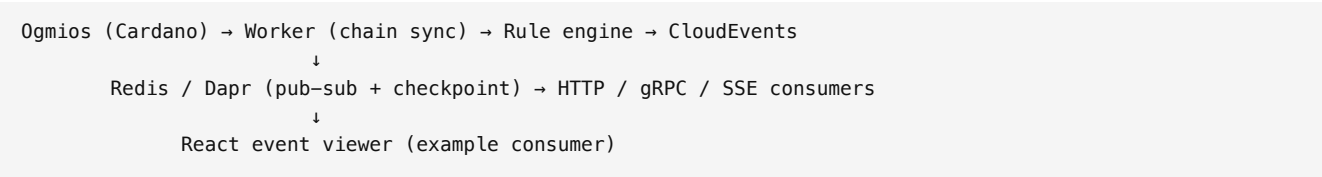
Solution

The project delivers a **rule-based filtering and event-emission engine** built on the [OgmiosDotnet](#) ecosystem:

1. **Chain sync** — A .NET Worker connects to Ogmios and processes live (or catch-up) blocks.
2. **Pluggable rules** — Configurable filters decide which transactions match. Built-in rules cover address match, policy ID / asset, metadata key/value, governance & treasury (including Conway treasury-withdrawal proposals), and an all-transactions catch-all. Custom rules can be added without rewriting the pipeline.
3. **Standardised events** — Matches are emitted as [CloudEvents 1.0](#) with Cardano-oriented extensions, so downstream systems share one schema.
4. **Dual delivery** — The same events are available via **HTTP** (Dapr pub/sub over Redis Streams), **gRPC** server-streaming, and **SSE** for browsers—so consumers need no Ogmios SDK and can be written in any language.
5. **Reliability** — At-least-once delivery with retry and dead-letter handling; Dapr state-store checkpointing with ETag concurrency for safe resume after restart.
6. **Interactive visualisation** — A React event-viewer UI demonstrates real-time consumption (Overview, Minswap demo haul, Live Feed, Consumers), proving that independent frontends can sit on the same delivery layer.

7. **Packaging for adoption** — Versioned Docker images on GHCR, Domain/Engine libraries on nuget.org, example configs (governance, treasury, metadata), Compose-based local stack (worker, Redis, Dapr, Grafana, viewers), and community docs (getting started, architecture, integration, CONTRIBUTING).

Architecture (at a glance)



Clean separation of **Domain**, **Engine**, and **Worker** keeps filtering logic reusable (NuGet) while the Worker and UI remain independently deployable.

Who it is for

- .NET and polyglot Cardano builders who want filtered chain events without owning full Ogmios client stacks
- DApps, dashboards, monitoring, and analytics that need address-, asset-, metadata-, or governance-focused streams
- Operators who want a Compose-friendly local stack and containerised deployment artefacts

Funding context

Delivered under **Project Catalyst Fund 14**, challenge **Cardano Open: Developers**, as proposal **OgmiosDotnet: Cardano Blockchain Events & Visualisation** (project #1400091), funded at **A100,000**, across four milestones from core filtering through delivery, visualisation, and community release.

List of challenge KPIs and how the project addressed them

Challenge: **Fund 14 — Cardano Open: Developers** (Developer Tools).

Challenge outcome	How this project addressed it
Open-source, reusable developer tooling	MIT-licensed public repo from day one; Domain + Engine published on nuget.org ; Worker + UI images on GHCR
Lower barriers to building on Cardano	Language-agnostic delivery (HTTP CloudEvents, gRPC, SSE) so consumers need no Ogmios SDK or WebSocket client
High-quality documentation & onboarding	docs/getting-started.md , architecture/event-schema/integration guides, example profiles, expanded CONTRIBUTING.md
Evidence of delivery & community visibility	Milestone PoAs under reports/ , tagged v1.0.0 , public demo video, and social launch post

List of project KPIs and how the project addressed them

Project KPI (from proposal / milestones)	Status	Evidence
Core filtering engine with pluggable rules	Met	5 built-in rules; custom-rule guide — M1 PoA
≥3 example rules against live block data	Met	Address, metadata, governance/treasury (+ PolicyIdAsset)
Standardised event schema (CloudEvents)	Met	docs/event-schema.md
Queue-backed delivery (HTTP + gRPC), at-least-once / retry	Met	Redis Streams + Dapr; DLQ — M2 PoA
Docker Compose local stack	Met	Root <code>docker-compose.yml</code> (worker, Redis, Dapr, Grafana, viewers)
React consumer with real-time visualisation	Met	Tabbed UI (Overview / Minswap / Live Feed / Consumers) — M3 PoA
≥2 rule configs demonstrated visually	Met	Governance/treasury + metadata examples; Minswap address-match demo

Project KPI (from proposal / milestones)	Status	Evidence
Public container images + versioned GitHub release	Met	GHCR <code>1.0.0</code> ; <code>v1.0.0</code>
Example configs (governance, treasury, metadata)	Met	<code>examples/</code> + <code>run-example.sh</code>
Community onboarding + close-out artefacts	Met	Getting started, CONTRIBUTING, this PCR, PCV link below
Unit tests passing	Met	80/80 tests — <code>test-report.md</code>

Key achievements (in particular around collaboration and engagement)

- Shipped a full path from **chain sync** → **rules** → **CloudEvents** → **UI**, so reviewers and adopters can run one Compose stack and see live filtered events.
- Published **reusable libraries** (Domain/Engine on nuget.org via Trusted Publishing) and **versioned images** so teams can embed or operate without forking the worker.
- Delivered a **close-out consumer showcase** (Minswap haul visualisation) that makes address-match filtering tangible for demos and social proof.
- Maintained milestone **proof-of-achievement** docs with pinned commits/tags for Catalyst review transparency.
- Engaged the community via public GitHub, YouTube demo material, and a [social launch post](#).

Key learnings

- Hosted Ogmios TLS/name mismatches need explicit HTTP handler policy — document early for adopters.
- Compose volume merges append rather than replace; `WORKER_APPSETTINGS` is the reliable way to swap example configs.
- Governance / treasury chain events are sparse on mainnet; demos should pair rare rules with high-frequency ones (metadata, DEX addresses).
- Multi-arch Docker under QEMU failed for `Grpc.Tools/protoc`; `v1.0.0` ships `linux/amd64` for reliable CI publish.
- Catalyst evidence works best with **stable URLs** (tags, release assets) rather than floating `main` links.
- Treasury-withdrawal matching depends on correctly mapping Conway `treasuryWithdrawals` governance proposals from Ogmios into `HasTreasuryWithdrawal` (fixed in the post-review hardening pass).

Next steps for the product or service developed

Focus is on **adoption and continuous improvement**, not a large new feature programme:

- Grow real-world adoption — onboarding, examples, and making the `v1.0.0` release easy for developers to run and embed
- Iterate on practical improvements driven by how people actually use the stack
- Stay in sync with **Ogmios** upgrades and related schema/client changes
- Listen to **community feedback** (GitHub issues, discussions, and contributor PRs) and prioritise accordingly
- Keep the MIT open-source project maintained so it remains a reliable building block for the Cardano ecosystem

Final thoughts/comments

The proposal asked for a simple way to filter Cardano blocks and receive only needed transactions over HTTP/gRPC, without Ogmios SDK lock-in. The delivered system meets that brief: open source, config-driven rules, dual delivery, observability, NuGet + containers, and a React consumer that proves the pipeline end-to-end. Remaining work is adoption and hardening, not missing core scope.

Links to other relevant project sources or documents

Item	Link
Source repository	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents
Catalyst proposal	https://projectcatalyst.io/funds/14/cardano-open-developers/ogmiosdotnet-cardano-blockchain-events-and-visualisation
GitHub Release v1.0.0	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/releases/tag/v1.0.0
Release workflow run	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/actions/runs/30199956555
Worker image (GHCR)	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/pkgs/container/ogmiosdotnet.blockchainevents
Event viewer image (GHCR)	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/pkgs/container/ogmiosdotnet.blockchainevents%2Fevent-viewer
NuGet Domain 1.0.0	https://www.nuget.org/packages/OgmiosDotnet.BlockchainEvents.Domain/1.0.0
NuGet Engine 1.0.0	https://www.nuget.org/packages/OgmiosDotnet.BlockchainEvents.Engine/1.0.0
Getting started	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/blob/main/docs/getting-started.md
Milestone PoAs	M1 , M2 , M3 , M4
Test report	test-report.md
Examples	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/tree/main/examples
Social media post	https://x.com/ItsDave_ADA/status/2083529870584226143
PCR PDF (Catalyst)	https://github.com/ItsDaveB/OgmiosDotnet.BlockchainEvents/blob/main/reports/milestone-4/close-out-report.pdf
Close-out video (PCV)	https://youtu.be/4B1VjrD4_Og
Technical demo (longer walkthrough)	https://youtu.be/-UUB0f4Dwfg

Link to Close-out video (PCV)

PCV (2–5 minutes, public YouTube): https://youtu.be/4B1VjrD4_Og

Dedicated Project Close-out Video covering challenge/approach, progress & KPIs, product demonstration (with pointer to the advanced technical demo), and next steps. Script: [close-out-video-script.md](#). The longer technical demo remains supplementary: <https://youtu.be/-UUB0f4Dwfg>.